

Catálogo + Carrito con checkout por WhatsApp

Arquitectura y código base de una tienda online 100% estática para negocios locales: sin base de datos tradicional, sin backend y sin pasarela de pago. El panel de administración es un documento de Google Sheets y el pedido se cierra por WhatsApp.

Dato	Valor
Stack	React 19 + Vite 8 + Tailwind CSS 4 + PapaParse
Base de datos	Google Sheets publicado como CSV (solo lectura)
Pagos	Efectivo al recibir o transferencia bancaria (fuera de la web)
Confirmación	wa.me/número?text=... con pedido estructurado
Demo online	https://valentinjurado.github.io/catalogo-whatsapp/ (catálogo de ejemplo)
Hosting	Vercel / Netlify / GitHub Pages (plan gratuito)
Costo mensual del negocio	Dominio (ej. .com.ar) — sin servidores ni licencias
Build de producción	315,7 kB JS (97,0 kB gzip) + 40,1 kB CSS (8,0 kB gzip)
Fecha del documento	22/09/2026

Idea central. El navegador del cliente hace un *fetch* al CSV del Google Sheets, arma el pedido en el carrito y, al confirmar, abre WhatsApp con el mensaje ya escrito. No hay servidor que mantener, no hay datos de tarjetas en juego y no existe superficie para inyección SQL porque no hay SQL.

1. Resumen del sistema

El sistema tiene dos pantallas de uso y una planilla:

- **Vista cliente:** catálogo con buscador y filtros, carrito lateral (off-canvas), checkout en 3 pasos y confirmación por WhatsApp.
- **Panel administrador:** el Google Sheets. El dueño del local carga, edita, oculta o borra productos sin tocar código. Los cambios se ven en la web al recargar la página.
- **Sin backend ni base de datos:** el sitio es un paquete de archivos estáticos. No hay API keys, no hay credenciales, no hay costos de infraestructura.

Flujo completo

CLIENTE	DUEÑO DEL LOCAL
-----	-----
1. abre la web	<---- CSV público <---- Google Sheets (productos)
2. elige productos	(precio, stock, oferta, foto)
3. elige entrega: envío a domicilio o retiro	
4. elige pago: efectivo o transferencia (si es transferencia ve alias/CBU + copiar)	
5. confirma ==>	wa.me + mensaje estructurado ==> WhatsApp del local

(con n° de orden, detalle, ==> prepara y responde
subtotales, entrega, pago)

El pago NUNCA pasa por la web: se coordina por fuera (efectivo al recibir
o transferencia bancaria con comprobante enviado por chat).

Por qué este diseño conviene al negocio

- **Costo casi nulo:** hosting gratuito + dominio. Nada de servidores, bases de datos ni suscripciones de pasarelas.
- **Cero riesgo financiero:** al no integrar cobros, no hay comisiones, ni cumplimiento PCI-DSS, ni riesgo de fraude con tarjetas.
- **Curva de administración nula:** el dueño ya sabe usar una planilla de cálculo; no hay que enseñarle un panel nuevo.
- **Rapidez:** el pedido llega a WhatsApp con todos los datos cargados, así el local solo confirma y prepara (menos ida y vuelta, menos pedidos mal tomados).
- **Publicación inmediata:** es un sitio estático: se copia a cualquier hosting y funciona.

2. Estructura del Google Sheet (base de datos y panel)

Una sola pestaña, con la fila 1 como encabezado. El nombre de las columnas es tolerante: no distingue mayúsculas ni tildes y acepta sinónimos (por ejemplo *título*, *titulo*, *nombre* o *producto*; *imagen*, *foto* o *url_imagen*).

Columna	Oblig.	Ejemplo	Qué hace / formato
id	no	MIL01	Identifica el producto. Si falta se genera desde el título. Debe ser único : el carrito agrupa por id.
titulo	Sí	Milanesa napolitana	Nombre visible. Una fila sin título se ignora.
descripcion	no	Con muzzarella y papas	Texto libre. En la tarjeta se recorta a 2 líneas.
precio	Sí	9800	Acepta 9800, 9.800,50, \$ 9.800. La coma decimal se interpreta sola.
precio_oferta	no	8500	Si es menor al precio, se muestra tachado y con el % de descuento.
categoria	no	Rotisería	Con esto se arman los filtros de arriba del catálogo.
url_imagen	no	https://...jpg	URL pública de la foto. En Drive: drive.google.com/uc?export=view&id=ID
stock	no	12	0 = Agotado (no se puede agregar). Vacío = sin control de stock.
unidad	no	kg	Se muestra como “/ kg” junto al precio.
etiquetas	no	Popular, Oferta	Hasta 2 etiquetas separadas por coma.
destacado	no	si	Los destacados se ordenan primero en el catálogo.
activo	no	si	Con “no” el producto no se publica (borrado lógico, reversible).
orden	no	1	Orden manual dentro de la categoría (1, 2, 3...).

Cómo publicarla para que la web la lea

- **Opción A (recomendada).** Archivo → Compartir → **Publicar en la web** → elegir la pestaña y el formato **CSV** → Publicar → copiar la URL y pegarla en `catalogo.hojaCsv`. La hoja queda de solo lectura pública.
- **Opción B.** Compartir como “cualquiera con el enlace puede ver” y pegar solo el ID del documento en `catalogo.hojaId` más el gid de la pestaña: la app arma el endpoint CSV de Google.
- **Validación previa.** El proyecto incluye un validador que usa el mismo parser que la web: `npm run validar <archivo.csv o URL>`. Avisa de filas sin precio, imágenes faltantes, ids repetidos y errores de comillas.

Reglas de datos que conviene respetar

- No repetir **id**: dos filas con el mismo id se comportan como un mismo producto en el carrito.
- Si un texto tiene **comas** (por ejemplo “Docena con membrillo, crema y dulce”), la celda debe ir entre comillas cuando se exporta a CSV — el validador lo detecta.
- Evitar cambiar el **id** de un producto existente: se pierde el historial del carrito guardado.
- Los precios son los que se muestran y con los que se calcula el total; la web nunca los inventa.

3. Servicio de obtención de datos (Sheets → JSON)

Vive en `src/services/catalogo.js`. No importa React: es una función pura sobre texto CSV y un *fetch*. Eso permite reusarla desde un script de Node (el validador usa exactamente esta función).

```
// 1) Origen de datos (src/config/negocio.js)
export function urlHojaCsv() {
  const { hojaCsv, hojaId, gid, modoDemo } = NEGOCIO.catalogo
  if (hojaCsv?.trim()) return hojaCsv.trim() // CSV publicado
  if (hojaId?.trim()) { // endpoint gviz
    return `https://docs.google.com/spreadsheets/d/${hojaId.trim()}/gviz/tq?tqx=out:csv&gid=${gid}`
  }
  return modoDemo ? `${import.meta.env?.BASE_URL ?? ''}productos-demo.csv` : ''
}

// 2) Descarga + normalización + caché (fragmento)
export async function obtenerCatalogo({ forzar = false } = {}) {
  const url = urlHojaCsv()
  if (!url) throw new Error('Falta configurar el Google Sheet en negocio.js')
  const cache = leerCache() // localStorage 'catalogo.cache.v1'
  if (cache && !forzar && !cacheVencido(cache)) return { ...cache, desdeCache: true }

  let res
  try { res = await fetch(url, { redirect: 'follow' }) }
  catch (e) { return cache ? { ...cache, desdeCache: true, avisoRed: true } : Promise.reject(e) }
  if (!res.ok) throw new Error(`El catálogo respondió ${res.status}`)

  const texto = await res.text()
  // Si la hoja no es pública, Google devuelve el HTML del login:
  if (/^<!doctype|html/i.test(texto)) throw new Error('La hoja no está publicada como CSV.')

  const datos = transformarCatalogo(texto)
  guardarCache({ ...datos, actualizado: new Date().toISOString(), guardadoEn: Date.now() })
  return { ...datos, desdeCache: false }
}
```

Normalización de cada fila (PapaParse → objeto de producto)

```
const { data } = Papa.parse(csv.trim(), {
  header: true, // primera fila = nombres de columna
  skipEmptyLines: 'greedy', // ignora las filas fantasma que agrega Sheets
  transformHeader: h => normalizarTexto(h).replace(/s+/g, '_'),
})

// Alias aceptados por columna: el dueño escribe como quiere
const ALIAS = {
  titulo: ['titulo', 'título', 'nombre', 'producto', 'artículo'],
  precio: ['precio', 'precio_lista', 'valor'],
  oferta: ['precio_oferta', 'oferta', 'precio_promo', 'precio_promocional'],
  imagen: ['url_imagen', 'imagen', 'foto', 'url_foto', 'url'],
  activo: ['activo', 'visible', 'publicado'],
  // ... categoria, descripcion, stock, unidad, etiquetas, destacado, orden
}

// Fila cruda -> producto canónico que consume React
{
  id, titulo, descripcion,
  precio, precioOferta, // número o null
  precioFinal, // oferta si existe y es menor
  categoria, urlImagen, unidad, etiquetas,
  stock, sinStock: stock === 0, // -1 = sin control de stock
  destacado, activo, orden,
}
```

Decisiones que hacen que esto no se rompa

- **Caché en localStorage con vencimiento (10 min por defecto):** la web abre al instante con el último catálogo guardado y refresca en segundo plano (stale-while-revalidate).
- **Si Google falla, la tienda sigue vendiendo:** se muestra el catálogo en caché y un aviso discreto. Nunca una pantalla en blanco.
- **Errores explicados para el humano:** “La hoja no está pública. Archivo > Compartir > Publicar en la web > CSV.”
- **Avisos no bloqueantes:** los productos sin precio válido se listan en un desplegable para que el dueño los corrija, sin romper el catálogo.
- **Parseo de precios tolerante:** 1.500,50 / \$1500 / 1,500.50 se interpretan correctamente; una oferta mayor al precio se descarta para no inflar el total.

4. Lógica del carrito y generación del mensaje de WhatsApp

Estado global con reducer (reglas del carrito en un solo lugar)

```
// src/state/carritoReducer.js — acciones: AGREGAR · CAMBIAR_CANTIDAD · QUITAR
// NOTA · VACIAR · SET_ENTREGA · SET_PAGO · SET_DATOS · SINCRONIZAR · HIDRATAR
case 'AGREGAR': {
  const existente = estado.items.find(i => i.id === accion.producto.id)
  if (existente) {
    // mismo producto: suma cantidad
    return { ...estado, items: estado.items.map(i => i.id === accion.producto.id
      ? { ...i, cantidad: Math.min(i.cantidad + (accion.cantidad || 1), MAX_POR_LINEA) } : i) }
  }
  return { ...estado, items: [...estado.items, lineaDesdeProducto(accion.producto,
    accion.cantidad || 1)] }
}

// SINCRONIZAR: cuando llega un catálogo nuevo, refresca precios y stock de las
// líneas ya cargadas (y elimina lo que se dio de baja) sin perder cantidades.
```

Totales, envío y validación (funciones puras, testeadas)

```
// src/services/pedido.js
export function calcularTotales(items, entrega = 'envio') {
  const lineas = items.map(item => ({
    ...item,
    precioUnitario: item.precioFinal ?? item.precio,
    subtotalLinea: (item.precioFinal ?? item.precio) * item.cantidad,
  }))
  const subtotal = lineas.reduce((a, l) => a + l.subtotalLinea, 0)
  const unidades = lineas.reduce((a, l) => a + l.cantidad, 0)
  const usaEnvio = entrega === 'envio' && NEGOCIO.entrega.envio.habilitado
  const envio = usaEnvio && !envioGratis(subtotal) ? NEGOCIO.entrega.envio.costos : 0
  return { lineas, subtotal, unidades, envio, total: subtotal + envio }
}

export function validarCheckout({ items, entrega, pago, datos }) {
  const errores = {}
  if (!items.length) errores.items = 'El carrito está vacío.'
  if (datos.nombre.trim().length < 3) errores.nombre = 'Ingresa tu nombre y apellido.'
  if (datos.telefono.replace(/\D/g, '').length < 8) errores.telefono = 'Teléfono inválido.'
  if (entrega === 'envio' && datos.direccion.trim().length < 5)
    errores.direccion = 'Ingresa la calle, número y localidad.'
  return { valido: Object.keys(errores).length === 0, errores } // la UI marca cada campo
}

export function generarNumeroOrden(prefijo = 'PED') { // PED-261004-7K3F
  // fecha (aammdd) + 4 caracteres de un alfabeto sin letras/números ambiguos
}
```

Generador del mensaje y del link (src/services/whatsapp.js)

```
export function construirMensajePedido(pedido, negocio = NEGOCIO) {
  const L = []
  L.push(`*NUEVO PEDIDO ${pedido.numeroOrden}*`) // *negrita* = marcado de WhatsApp
  L.push(`${negocio.marca.nombre} — ${fechaLegible(pedido.fecha)}`)
  L.push('', '*Detalle del pedido*')
  pedido.items.forEach(i => {
    L.push(`• ${i.cantidad} x ${i.titulo}${i.unidad ? ` (${i.unidad})` : ''} — ${formatearPrecio(i.subtotalLinea)}`)
    if (i.nota) L.push(`■ Nota: ${i.nota}`) // "sin cebolla", etc.
  })
  L.push('', `Subtotal (${pedido.unidades} items): ${formatearPrecio(pedido.subtotal)}`)
  if (pedido.entrega === 'envio') {
    L.push(pedido.envio > 0 ? `Envío: ${formatearPrecio(pedido.envio)}` : 'Envío: sin cargo')
  }
  L.push(`*TOTAL: ${formatearPrecio(pedido.total)}*`)
}
```

```
L.push('', '*Entrega*', `Modalidad: ${pedido.entregaTexto}`)
if (pedido.entrega === 'envio') L.push(`Dirección: ${pedido.cliente.direccion}`)
else L.push(`Retiro en: ${negocio.entrega.retiro.direccion}`)
L.push('', '*Pago*', `Forma de pago: ${pedido.pagoTexto}`)
if (pedido.pago === 'transferencia') {
  L.push(`Alias: ${negocio.pago.transferencia.alias}`)
  L.push(negocio.pago.transferencia.confirmacion + '.') // frase exigida por el flujo
} else {
  L.push(negocio.pago.efectivo.detalle)
}
L.push('', '*Cliente*', `Nombre: ${pedido.cliente.nombre}`,
  `Teléfono: ${pedido.cliente.telefono}`)
return L.join('\n')
}

export function construirUrlWhatsapp(mensaje, numero = NEGOCIO.whatsapp.numero) {
  return `https://wa.me/${limpiarTelefono(numero)}?text=${encodeURIComponent(mensaje)}`
}
```

Mensaje real generado durante la verificación

```
*NUEVO PEDIDO PED-260922-HKFL*
Almacén Doña Rosa — 22/09/2026 22:39

*Detalle del pedido*
• 1 x Milanesa napolitana con guarnición (porción) — $ 8.500,00
• 1 x Empanadas de carne cortada a cuchillo (media docena) — $ 6.500,00
• 1 x Pollo al spiedo (unidad) — $ 11.500,00

Subtotal (3 ítems): $ 26.500,00
Envío: sin cargo (promoción)
Ahorro por ofertas: -$ 2.000,00
*TOTAL: $ 26.500,00*

*Entrega*
Modalidad: Envío a domicilio
Dirección: Rivadavia 1234, Tandil
Referencia: Porton negro, timbre 2
Horario preferido: entre 20 y 21 hs

*Pago*
Forma de pago: Transferencia bancaria
Alias: almacen.donarosa.mp
Ya realicé el pago al alias indicado, te adjunto el comprobante.

*Cliente*
Nombre: Valentin Jurado
Teléfono: 2494 123456
```

La frase “Ya realicé el pago al alias indicado, te adjunto el comprobante” se incluye automáticamente cuando el cliente elige transferencia, para que el local sepa que debe esperar el comprobante en el chat.

Pantallas del flujo de pago (sin datos de tarjeta)

- **Paso 1 – Datos y entrega:** nombre, teléfono, dirección (solo si elige envío), referencia para el repartidor, horario preferido y aclaraciones. Validación en vivo por campo.
- **Paso 2 – Forma de pago:** Efectivo (se abona al recibir) o Transferencia. Al elegir transferencia se muestra una ficha con Alias, CBU/CVU, titular, banco, CUIT y el importe exacto, cada dato con botón *Copiar*, más los 3 pasos a seguir y un check de conformidad. No hay ningún campo de tarjeta en toda la aplicación.
- **Paso 3 – Confirmar:** resumen del pedido con número de orden y totales; el botón es un enlace real a `wa.me` (si el navegador bloquea ventanas emergentes, el enlace igual funciona).
- **Cierre:** pantalla de éxito con el número de orden, botón para reabrir WhatsApp y opción de copiar el resumen como comprobante del armado.

5. Estructura del proyecto

```

catalogo-whatsapp/
├── public/
│   ├── productos-demo.csv          catálogo de ejemplo (modo demostración)
│   ├── favicon.svg
│   └── docs/
│       ├── plantilla-google-sheet.csv  plantilla lista para importar en Sheets
│       ├── ARQUITECTURA.pdf          este documento
│       └── generar-pdf-arquitectura.py  generador de este PDF
├── scripts/
│   ├── validar-catalogo.mjs          valida tu CSV con el mismo parser de la web
│   ├── test-servicios.mjs            14 pruebas de la lógica pura (sin navegador)
│   ├── cdp.mjs                      conexión CDP reutilizable (capturas y E2E)
│   ├── e2e.mjs                      flujo de compra completo: 35 comprobaciones
│   └── capturar.mjs                  capturas de pantalla para la ficha de venta
├── src/
│   ├── config/
│   │   ├── negocio.js               ■■■ ÚNICO archivo que se edita por cliente
│   │   ├── paletas.js               6 paletas de color aplicadas en runtime (CSS vars)
│   │   └── services/                lógica sin React, testeable
│   │       ├── catalogo.js          CSV de Sheets → JSON (+ caché + errores claros)
│   │       ├── pedido.js            totales, envío, n° de orden, validaciones
│   │       ├── whatsapp.js          mensaje estructurado + link wa.me
│   │       └── formato.js           precios, fechas, normalización de texto
│   ├── state/
│   │   ├── carritoReducer.js        reglas del carrito (puro)
│   │   └── CarritoContext.jsx        estado global + persistencia en localStorage
│   ├── hooks/
│   │   ├── useCatalogo.js          carga con caché y refresco en segundo plano
│   │   └── useCopiar.js             copiar alias/CBU con respaldo y aviso de fallo
│   ├── components/
│   │   ├── layout/                 Header · Hero · Footer
│   │   ├── catalogo/              Catalogo · Filtros · ProductCard
│   │   ├── carrito/               CartWidget (off-canvas) · ItemCarrito · SelectorEntrega · BarraPedidoMovil
│   │   ├── checkout/              CheckoutModal · PasoDatos · PasoPago · DatosTransferencia
│   │   │   ├── PasoConfirmar · PantallaExito
│   │   └── ui/                    Boton · Modal · Toast · Campo · ImagenProducto · Estados · Iconos
│   ├── App.jsx                    composición de la página
│   └── main.jsx                    tema, SEO y providers

```

Dos reglas de oro para que sea mantenible y vendible

- **services/ nunca importa React y components/ nunca hace fetch.** Los componentes reciben datos y emiten acciones; la red y el almacenamiento viven en hooks y servicios. Así la lógica del negocio se puede testear sin navegador (ya hay 14 pruebas).
- **Todo lo que cambia de un local a otro está en un solo archivo** (src/config/negocio.js): marca, colores, número de WhatsApp, envío, alias/CBU, textos y campos del checkout. Re-marcar la plantilla no requiere tocar componentes ni recompilar Tailwind (los colores se inyectan como variables CSS).

Reutilización por cliente (white-label en minutos)

Qué se cambia	Dónde	Efecto
Nombre, eslogan, logo	marca.*	Encabezado, pie, título de la página (SEO)
Paleta y redondeo	marca.tema / marca.radio	6 paletas listas: verde, azul, bordo, naranja, violeta, grafito
Número de WhatsApp	whatsapp.numero / nombreVendedor	Destino de todos los pedidos y saludo del mensaje

Qué se cambia	Dónde	Efecto
Origen de productos	catalogo.hojaCsv o hojald+gid	Se conecta el Sheet del cliente
Envío	entrega.envio.* / entrega.retiro.*	Costo, envío gratis desde X, zona, demora, dirección y horarios
Cobro	pago.efectivo / pago.transferencia	Alias, CBU, titular, banco y textos del aviso
Formulario	pedido.campos.*	Qué se le pide al cliente (dirección solo si hay envío)
Todos los textos	textos.*	Portada, avisos y pie, sin buscar en componentes

6. Seguridad y rendimiento

Superficie de ataque del sistema

Riesgo clásico	¿Aplica?	Por qué
Inyección SQL	No	No existe base de datos ni consultas: los datos son un CSV público de lectura.
Robo de credenciales	No	No hay login, ni tokens, ni API keys en el front. La hoja solo se lee.
Datos de tarjetas (PCI)	No	No se piden ni se procesan medios de pago en la web.
Pagos fraudulentos	No	No hay pasarela: el cobro se acuerda por fuera y lo controla el local.
XSS desde la planilla	Mitigado	React escapa todo texto interpolado; no se usa innerHTML ni dangerouslySetInnerHTML.
Manipulación de precios	Mitigado	El total se recalcula desde los precios del catálogo; el mensaje lleva el detalle para que el local lo verifique.
Spam de pedidos	Bajo	No hay endpoint que atacar: el peor caso es un mensaje de WhatsApp, que el local puede bloquear.
Caída de Google Sheets	Mitigado	Caché local del último catálogo: la tienda sigue mostrando productos.

Rendimiento

- **Peso real medido:** 315,7 kB de JavaScript (97,0 kB gzip) y 40,1 kB de CSS (8,0 kB gzip). Una sola petición de datos: el CSV del Sheet (el resto queda en caché del navegador).
- **Sin dependencias pesadas:** solo React, Tailwind y PapaParse. Cero librerías de UI, de iconos o de estado (los iconos son SVG inline).
- **Carga diferida:** las 4 primeras imágenes del catálogo se cargan en modo *eager* con alta prioridad (mejora el LCP) y el resto en *lazy*.
- **Mobile-first:** una columna en celular, grilla de 2 a 4 columnas desde tablet/desktop; el carrito es un panel lateral y en el celular una barra fija inferior.
- **Accesibilidad:** foco visible, cierre con Escape, foco atrapado en el modal y en el panel, etiquetas asociadas a cada campo y respeto de "reducir movimiento".
- **Sin jerga de servidor:** es un paquete estático; se sirve desde CDN en cualquier hosting.

Cómo es el arranque en frío, paso a paso

```

npm install          # instala React, Vite, Tailwind y PapaParse
# 1. editar src/config/negocio.js (marca, whatsapp, envío, pago, hoja)
npm run validar public/productos-demo.csv # valida la planilla del cliente
npm run lint         # oxlint: 0 avisos esperados
npm test             # 14 pruebas de pedido, totales, parseo y mensaje
npm run dev          # desarrollo en http://localhost:5173
npm run build        # genera dist/ (paquete estatico listo para publicar)
npm run preview      # sirve dist/ localmente (puerto 4173)
npm run e2e          # flujo de compra completo en navegador: 35 comprobaciones
node scripts/capturar.mjs http://localhost:4173 docs/captura.png 1440 900 0 grilla

```

Publicación con hosting gratuito

- **Vercel:** `npx vercel deploy --prod` (framework Vite, salida dist). Dominio propio desde el panel.
- **Netlify:** arrastrar la carpeta `dist/` al panel, o `npx netlify deploy --prod --dir=dist`.
- **GitHub Pages:** subir el contenido de `dist` a la rama `gh-pages`. El proyecto ya compila con rutas relativas, así que también funciona en subcarpetas de usuario.

- **Dominio:** el negocio solo paga el dominio (por ejemplo .com.ar) y lo apunta al hosting. HTTPS queda incluido y es requisito para el portapapeles moderno (el botón Copiar tiene respaldo para contextos sin HTTPS).

7. Verificación realizada sobre este código

Todo lo que sigue se ejecutó sobre el código entregado, no es una descripción teórica.

Prueba	Resultado
Linter (oxlint)	OK — 0 avisos en src/ y scripts/
Compilación de producción	OK — 50 módulos, build en ~0,5 s
Pruebas de lógica (npm test)	OK — 14/14 (precios, totales, envío, validaciones, nº de orden, mensaje, link)
Validador del CSV (npm run validar)	OK — 20 filas, 20 productos activos, 5 categorías, 0 problemas
Fotos del demo	26 URLs verificadas (HTTP 200); 20/20 cargan y se pintan (600×450)
E2E navegador real (npm run e2e)	OK — 35/35 comprobaciones, repetible entre corridas
Link de WhatsApp	OK — wa.me/549249... con mensaje codificado completo, sin espacios sin codificar
Persistencia	OK — el carrito y el número de orden sobreviven al recargar (localStorage)
Responsive	OK — capturas en 1440×900 y 390×844 con barra de pedido fija en el celular

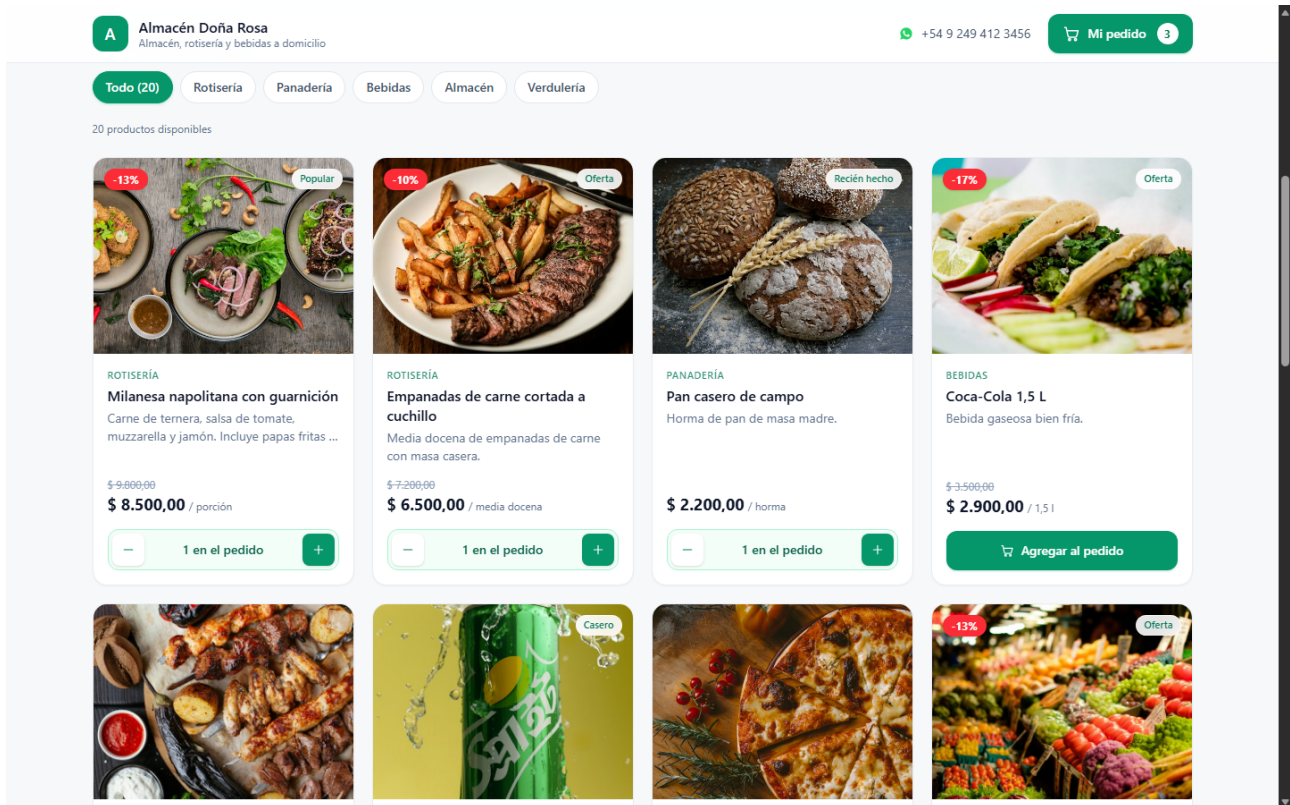
El E2E vive en `scripts/e2e.mjs` y escribe su informe en `docs/e2e-resultado.json`: recorre catálogo, búsqueda, filtros, carrito, checkout, transferencia, enlace de WhatsApp, pantalla de éxito y persistencia, y sale con código de error si alguna comprobación falla (se puede usar en CI).

Chequeos funcionales verificados en el navegador

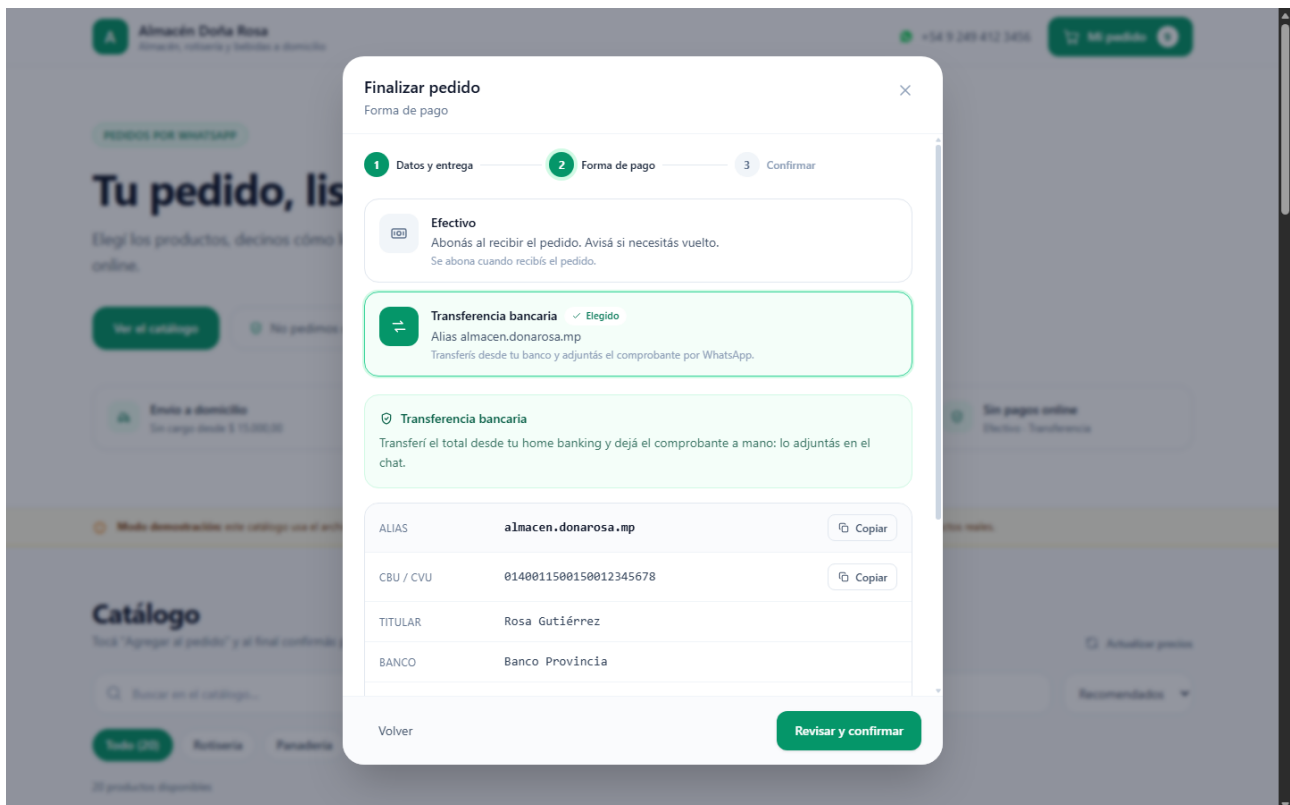
- El buscador filtra por título, descripción, categoría y etiquetas, sin distinguir tildes.
- Los filtros por categoría se generan solos desde la planilla (Rotisería, Panadería, Bebidas, Almacén, Verdulería en el ejemplo).
- Un producto con stock 0 se muestra como **Agotado** y no se puede agregar.
- Las ofertas muestran el precio anterior tachado y el porcentaje de descuento.
- El carrito suma cantidades del mismo producto, permite aclaraciones por línea ("sin cebolla") y calcula subtotal, envío y total.
- El envío gratis se aplica automáticamente al superar el monto configurado (y el carrito muestra cuánto falta para alcanzarlo).
- La validación del checkout marca cada campo con su error y no deja avanzar con datos incompletos.
- La pantalla de transferencia muestra Alias/CBU/titular/importe con botón Copiar (y avisa si el navegador bloquea el portapapeles).
- El paso final es un enlace real a wa.me: si el navegador bloquea ventanas emergentes, el usuario igual puede enviar el pedido.
- Al cerrar el flujo queda el número de orden visible y la opción de copiar el resumen.

Capturas del sistema funcionando

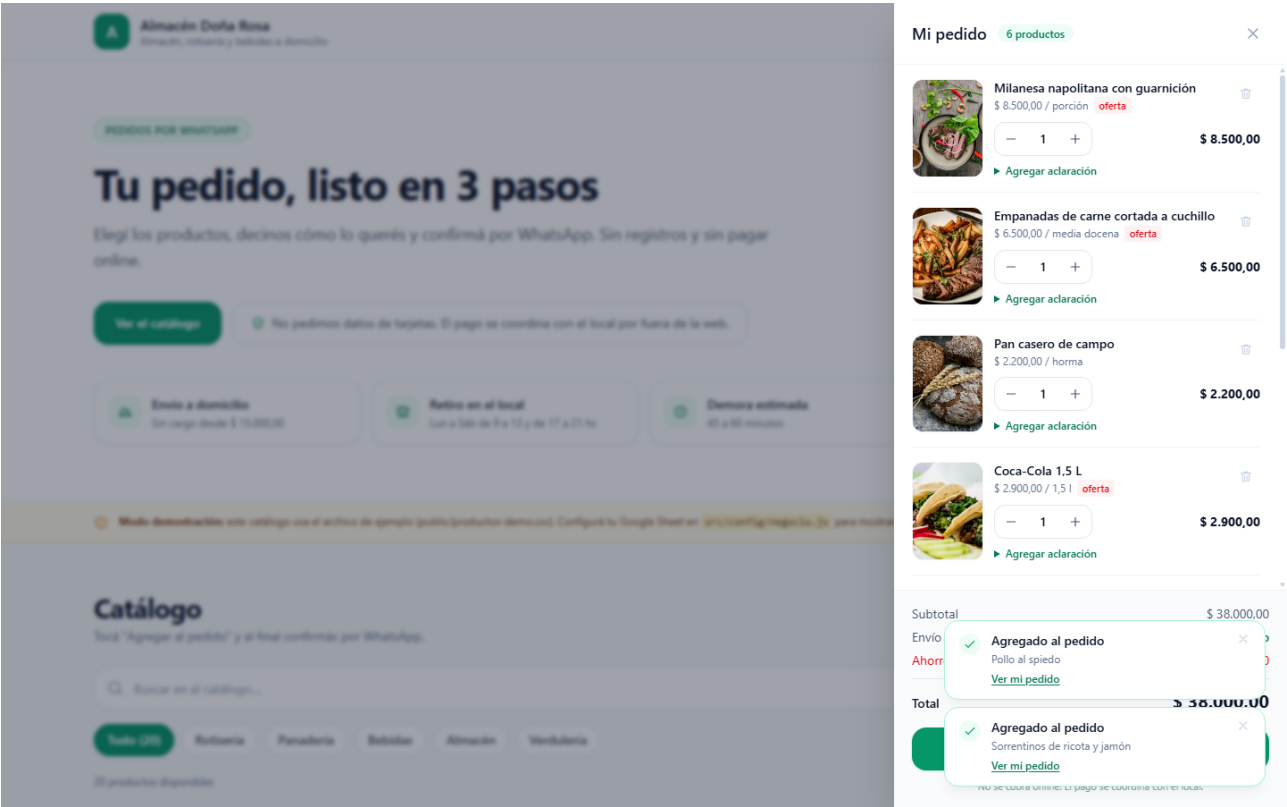
Catálogo en escritorio (1440 px): filtros generados desde la planilla, ofertas con precio tachado y porcentaje, etiquetas y botón de agregar por tarjeta.



Paso 2 del checkout: forma de pago. Al elegir transferencia aparecen Alias, CBU/CVU, titular, banco y el importe exacto, cada dato con botón Copiar. No existe ningún campo de tarjeta en toda la aplicación.



Carrito lateral (off-canvas): cantidades, aclaración por producto, modalidad de entrega, barra de progreso hacia el envío gratis y totales.



Versión móvil (390 px): la barra inferior fija mantiene el pedido a un toque de distancia. La vista está pensada mobile-first.



Nota sobre las capturas: se tomaron con un navegador headless (sin ventana). En ese entorno las transiciones CSS no progresan y las imágenes con carga diferida no se descargan; el script scripts/capturar.mjs fuerza ambas cosas solo para

poder fotografiar la página. El estado real se verificó midiendo el DOM (20/20 imágenes descargadas y visibles) y no confiando en la captura.

8. Límites conocidos y cómo venderlo como plantilla

Límites por diseño (conviene decirlos antes de vender)

- No hay stock en tiempo real ni reserva de mercadería: el local confirma por WhatsApp.
- El catálogo se actualiza cuando el cliente recarga la página (la caché dura 10 minutos).
- El *id* de cada producto debe ser único y estable: es lo que identifica la línea del carrito.
- Los pedidos no quedan registrados en ningún sistema; el historial es el chat de WhatsApp.
- Para ver el modo demostración hay que servir el sitio (`dev/preview/hosting`): abrir el archivo con doble clic no permite leer el CSV por restricciones del navegador.
- Los precios son responsabilidad de la planilla: la web muestra exactamente lo que dice la hoja.

Checklist de entrega por cliente

- Cargar marca (nombre, eslogan, tema, logo) en `negocio.js`.
- Cargar el número de WhatsApp real y probar un pedido de punta a punta desde un celular.
- Configurar envío (costo, envío gratis desde, zona, demora) y datos de retiro.
- Verificar alias/CBU/titular con el dueño y hacer una transferencia de prueba.
- Importar `docs/plantilla-google-sheet.csv` y cargar los productos reales.
- Publicar la hoja como CSV, pegar la URL y correr el validador.
- Reemplazar el favicon y revisar los textos de portada.
- Publicar en el hosting y entregar la URL + una guía de una página para el dueño.

Qué se puede cobrar por esto

- **Como servicio:** implementación por cliente (marca + planilla + publicación + capacitación de 20 minutos). Es un trabajo de horas, no de semanas, porque la plantilla ya está hecha.
- **Como plantilla:** venta del código con licencia de uso por proyecto, con la guía de instalación y las 6 paletas incluidas.
- **Ingreso recurrente:** mantenimiento (nuevos productos, cambios de precios masivos, promociones) y hosting/dominio a nombre del cliente.

Resumen en una frase. Es una tienda que no necesita servidor: el catálogo vive en una planilla de Google, el carrito vive en el navegador del cliente y el pedido viaja por WhatsApp. El negocio solo paga el dominio y el desarrollo se entrega en horas, no en semanas.